LocalServe

LOCAL SERVICES MARKETPLACE

System Technical Documentation Suite • Version 1.0.0

Prepared for: Academic Instructor Review & Development Team Reference

Project Metadata & Development Team

Repository Link: `git@github.com:abrshiz/Local-Services-Marketplace.git`

#	GROUP MEMBER NAME	STUDENT ID / ID
1	Abrham Wendesen	DDU1600055
2	Makbel Hailu	DDU1600488
3	Yeabsira Eskinder	DDU1600747
4	Eyob Jira	RMD940
5	Wegen Geremew	DDU1601938

1. Software Requirements Specification (SRS)

The LocalServe system supports two primary authenticated user roles. Session authentication is implemented via JSON Web Tokens (JWT) stored client-side.

1.1 Platform Roles & Permissions

ROLE	CORE PURPOSE	FUNCTIONAL CAPABILITIES & PERMISSIONS
Customer	End-users seeking professional home and local services.	• Browse categories and dynamically search for nearby providers. • Filter providers by rating, service category, price, and distance. • Create bookings and select preferred available date/time slots. • Track booking lifecycles in real-time. • Engage in private messaging with service providers. • Submit ratings and reviews for completed services. • Manage multiple saved addresses and card/cash payments.
Service Provider	Professionals offering on-demand services.	• Register, create profiles, and select specific work categories. • Define service titles, detailed descriptions, and base prices. • Manage operational availability status and time slots. • Accept, decline, and monitor customer booking requests. • Access an earnings overview and scheduled tasks dashboard. • Engage in real-time chat with clients concerning active orders.

1.2 Visual Interface & Feature Mapping

Based on the official **LocalServe user interface mockups**, the mobile client implements five high-fidelity feature interfaces:

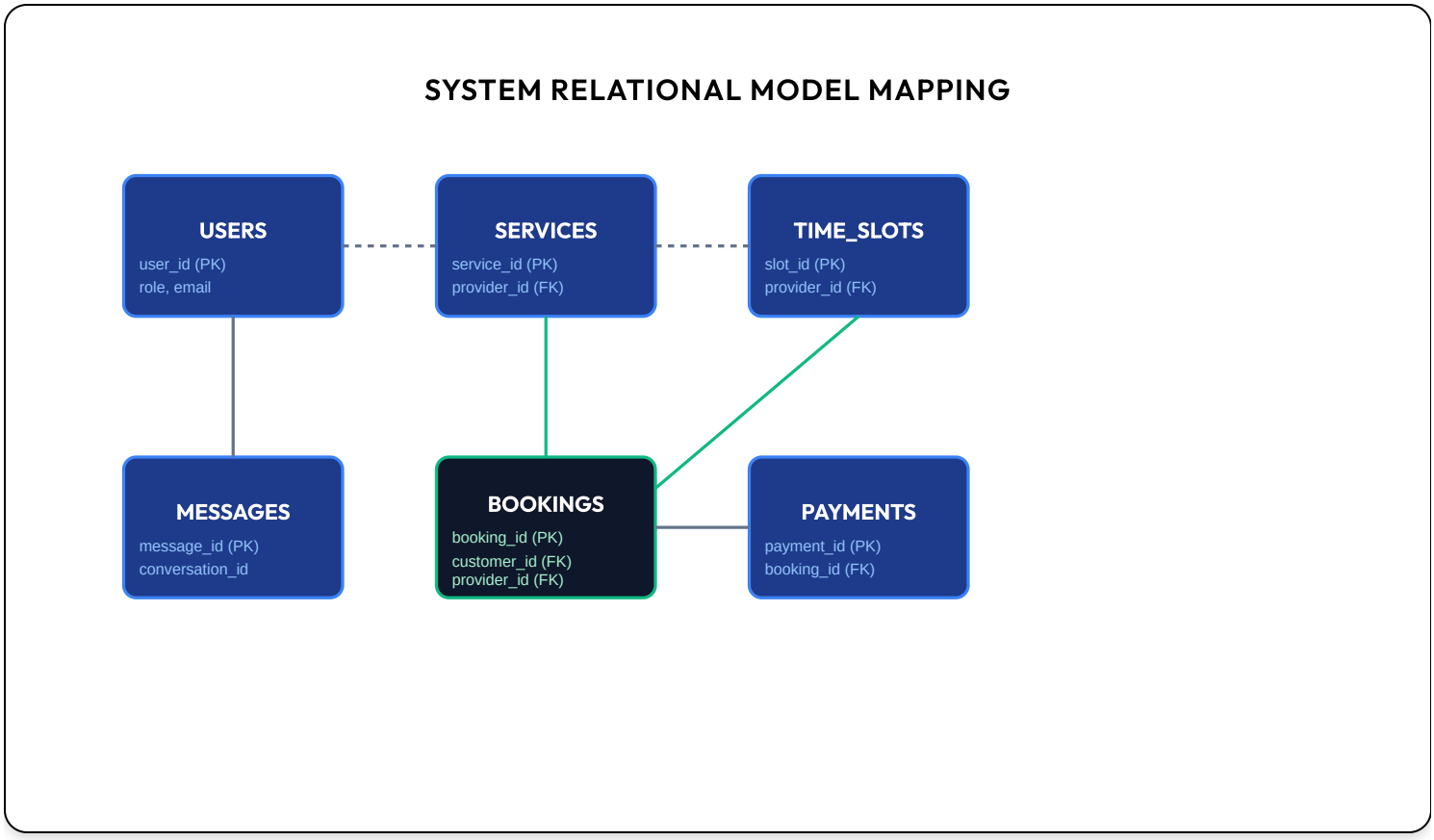
- **Authentication & Onboarding Screen (`sign_up_login_screen`)**: Form fields for both login and registration. Role selections (Customer vs. Provider) dynamically toggle fields like professional category selections.
- **Customer Discovery & Home Screen (`home_screen`)**: Displays location headers parsed from GPS sensors, category dynamic filters, and listings of local professionals with active badges (available in green, busy in orange).
- **Booking Details & Flow Screen (`booking_screen`)**: Integrates custom scheduling calendars (powered by `table_calendar`) with slot management, custom instructions, and pricing totals.
- **Real-time Messaging Screen (`messages_screen` / `chat_screen`)**: Visual list of active conversations, dynamic unread badge counts, and double-check read/receive indicators.
- **Profile Settings & Availability Console (`profile_screen`)**: A settings hub displaying user statistics and (for providers) the availability switches, category setups, and schedule panels.

2. Database Schema & Architecture (MySQL)

The LocalServe database relies on an optimized, highly structured relational layout built on **MySQL (v8.0+)**. The official database is named **`Local Market Place`**. Primary keys are dynamically handled via standard UUID strings. Foreign key constraints are strictly enforced using the transaction-safe **InnoDB** storage engine, and high performance is guaranteed via indexed lookup trees.

2.1 Relational Layout Structure

The entity relation mappings decouple the core structures, linking **Users** to nested addresses, skills, scheduling time slots, bookings, payments, and private chat registers.



2.2 Production MySQL DDL Script

The database schema is declared using standard MySQL DDL. It initializes the **`Local Market Place`** namespace and sets up 12 relational tables:

```sql
-- Create and activate the master team database
CREATE DATABASE IF NOT EXISTS `Local Market Place`;
USE `Local Market Place`;

CREATE TABLE IF NOT EXISTS users (
  user_id VARCHAR(36) PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  phone VARCHAR(50),
  role VARCHAR(50) NOT NULL,
  created_at DATETIME NOT NULL,
  updated_at DATETIME NOT NULL,
  bio TEXT,
  average_rating DECIMAL(3, 2) DEFAULT 0.00,
  is_verified TINYINT(1) DEFAULT 0,
  is_active TINYINT(1) DEFAULT 1,
  latitude DECIMAL(10, 8),
  longitude DECIMAL(11, 8),
  default_address_id VARCHAR(36),
  loyalty_points INT DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS addresses (
  address_id VARCHAR(36) PRIMARY KEY,
  user_id VARCHAR(36) NOT NULL,
  label VARCHAR(100),
  street VARCHAR(255),
  city VARCHAR(100),
  state VARCHAR(100),
  country VARCHAR(100),
  postal_code VARCHAR(20),
  latitude DECIMAL(10, 8),
  longitude DECIMAL(11, 8),
  is_default TINYINT(1) DEFAULT 0,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE IF NOT EXISTS services (
  service_id VARCHAR(36) PRIMARY KEY,
  provider_id VARCHAR(36) NOT NULL,
  category_id VARCHAR(36) NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  price_type VARCHAR(50) NOT NULL,
  base_price DECIMAL(10, 2) NOT NULL,
  is_active TINYINT(1) DEFAULT 1,
  FOREIGN KEY (provider_id) REFERENCES users(user_id) ON DELETE CASCADE,
  FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS bookings (
  booking_id VARCHAR(36) PRIMARY KEY,
  customer_id VARCHAR(36) NOT NULL,
  provider_id VARCHAR(36) NOT NULL,
  service_id VARCHAR(36) NOT NULL,
  slot_id VARCHAR(36),
  scheduled_time DATETIME NOT NULL,
  end_time DATETIME NOT NULL,
  status VARCHAR(50) NOT NULL,
  total_price DECIMAL(10, 2) NOT NULL,
  address_id VARCHAR(36),
  notes TEXT,
  service_started_at DATETIME,
  service_completed_at DATETIME,
  FOREIGN KEY (customer_id) REFERENCES users(user_id) ON DELETE RESTRICT,
  FOREIGN KEY (provider_id) REFERENCES users(user_id) ON DELETE RESTRICT,
  FOREIGN KEY (service_id) REFERENCES services(service_id) ON DELETE RESTRICT,
  FOREIGN KEY (slot_id) REFERENCES time_slots(slot_id) ON DELETE SET NULL,
  FOREIGN KEY (address_id) REFERENCES addresses(address_id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3. System Architecture & Backend Design

The Express backend application adopts a standard, clean multi-tier structure to decouple logical tasks.

3.1 Backend Architectural Layers

- **Routing & Core App (`src/index.js`)**: Starts the server, coordinates endpoints, registers routing structures, and parses JSON input streams.
- **Security & Authentication Middleware (`src/middleware/auth.js`)**: Intercepts secure HTTP requests. Ensures the `Authorization: Bearer <token>` header is present and decodes the JWT payload, extracting the `userId` for use in downstream controllers.
- **Data Mappers (`src/userMapper.js`)**: Resolves MySQL query structures into nested, standardized JSON models matching the exact entities defined in the Flutter application.
- **Database Pool Driver (`src/db.js`)**: Interacts directly with the MySQL Server using connection pools provided by the high-performance `mysql2` package.

3.2 Security Protocols

> **Cryptographic Cryptography & Hashing**
>
> Passwords are stored using **bcryptjs** with 10 salt rounds. Sessions are verified via cryptographically signed JWT strings passed in standard request headers, preventing SQL injection and cross-tenant data leaks.

4. REST API Specifications

All HTTP endpoints require JSON payloads and return descriptive envelopes.

4.1 REST API Endpoint Directory

METHOD	ENDPOINT ROUTE	AUTH	FUNCTIONAL SCOPE
POST	`/api/v1/auth/login`	No	Authenticates credentials; yields token & profile.
POST	`/api/v1/auth/register`	No	Creates user; yields active token.
GET	`/api/v1/providers/nearby`	Yes	Locates nearby providers sorted by GPS distance.
POST	`/api/v1/bookings`	Yes	Creates a new booking on an open time slot.

4.2 Key Payloads

User Authentication Response (Success: 200 OK)

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "userId": "usr_customer_1",
    "name": "Alex Customer",
    "email": "customer@demo.com",
    "phone": "+15551234",
    "role": "CUSTOMER",
    "averageRating": 0,
    "isVerified": 0,
    "isActive": 1
  }
}
```

5. Flutter Client Architecture

The mobile front-end application is built with **Flutter (Dart)**, utilizing **Clean Architecture** patterns combined with **BLoC** for reliable state isolation and UI management.

5.1 Three-Layer Architectural Decoupling

- **Presentation Layer (`lib/presentation/`)**: Declarative widget hierarchies responding to dynamic states emitted by feature BLoC files.
- **Domain Layer (`lib/domain/`)**: Houses pure Dart entities and abstract repository definitions, making the business logic highly testable and independent of any networking library.
- **Data Layer (`lib/data/`)**: Manages API communication using a global **Dio** client and local persistent storage.

5.2 Front-end Core Dependencies

- `flutter_bloc`: Emitted states handle load spinners, network success, and error alerts globally.
- `get_it`: Resolves repository instances synchronously without boilerplate parameters.
- `dio`: High-performance HTTP client supporting global logging interceptors and automatic timeout defaults.

6. Installation & Deployment Guide

Follow these steps to launch the LocalServe backend and Flutter mobile applications locally.

6.1 Quick-Start Commands

1. Node.js Backend setup

Requires Node.js (v18+) and an active MySQL Server (v8.0+).

```
$ cd backend
$ npm install
$ npm run seed      # Seeds master database Local Market Place
$ npm run dev       # Launches http://localhost:3000
```

Database connection details are defined in `backend/.env`:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=yourpassword
DB_NAME=Local Market Place
```

2. Flutter Client setup

```
$ flutter pub get
$ flutter run
```

6.2 Build Environment Scenarios

SCENARIO	RUN FLAG / COMMAND	FUNCTIONAL SCOPE
Emulator Setup	`flutter run --dart-define=API_BASE_URL=http://10.0.2.2:3000`	Routes client calls to the local development machine from inside Android virtual devices.
Offline Mock	`flutter run --dart-define=USE_MOCK_API=true`	Bypasses the internet connection and launches using mock datasets.
Release Build	`flutter build apk --release`	Assembles the production-ready Android package.

7. Testing & Verification Protocol

To run functional tests and verify role-based flows, use the pre-seeded credentials below:

7.1 Pre-seeded Demo Accounts

TESTING ROLE	USERNAME / EMAIL	PASSWORD	INTENDED TEST FLOW SCOPE
Customer	`customer@demo.com`	`password123`	Test the search, booking creation, card payment simulation, chat, and review submission flows.
Provider A	`provider1@demo.com`	`password123`	Log in as **Jordan Clean Co.** to manage bookings, adjust pricing, toggle availability, and reply to chat.
Provider B	`provider2@demo.com`	`password123`	Log in as secondary provider **Alex Provider** to check scheduling, isolation, and message routing.

Final Verification Checklist

1. Sanity Check MySQL Server is active and `Local Market Place` database is created.
2. Run `npm run seed` to verify all 12 tables are initialized and seeded.
3. Ensure Maps key in `AndroidManifest.xml` is restricted to `com.localservicemarket.localservicemarket` in Google Console.